SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

1. Introduction **3**
1.1. Purpose 3
1.2. Background 4
1.3. Target Stakeholder 5
2. System Architecture & Design **7**
2.1. High Level Architecture 7
2.1.1. Overview 7
2.1.2 LLM as Service Layer 7
2.1.3 Dependency Diagram 7
2.2.2. Sequence Diagram 8
1.3. Model Process: Machine Request Workflow 9
2.2. Technological Stack 9
2.2.1. Frontend 9
2.2.2. Backend 10
2.2.3. Database 10
2.2.4. Intelligence Layer (LLM) 10
2.2.5. Cloud/Deployment 10
2.3. Key Data Flows 10
2.3.1. Data Flow Diagram (DFD) 10
3. Functional Requirements & Scope **12**
3.1. Minimum Viable Product: 12
3.1.1. Core Features (3-5) 12
3.2. Non-Functional Requirements (NFRs) 13
3.3. Out of Scope / Future Enhancements 14
4. Monitor, Evaluation, Assumptions & Dependencies **14**
4.1. Technical Evaluation 14
4.1.1. Grayscale Rollout & A/B Testing 14
4.1.2. Emergency Rollback (ER) & Golden Release 14
4.1.3. Priority Matrix 15
4.2. Monitoring 15
4.2.1. System Health & Monitoring Plan 15
4.3. Assumptions 16
4.4. External Dependencies 16
5. Project Management & Team Contributions **17**
5.1. Project Timeline 17
5.2. Team Members Role 18
5.3. Recommendations 19

© UMHackathon 2026

1. Introduction

This section introduces OpsResolve, an AI-integrated strategic solution designed to resolve the operational inefficiencies and communication silos identified in the manufacturing and logistics workflow.

1.1. Purpose

The system analysis document highlights the technical scope and design-related decisions behind the OpsResolve development. The key elements covered are as follows:

- Architecture:
  It is a hybrid-cloud platform utilizing a decoupled architecture. The frontend is built on React (Vite) for rapid user interface responsiveness, while the backend is an Express.js environment. Data persistence and real-time synchronization are handled by Firebase, and cognitive processing is centralized through Google Gemini AI.
- Data Flows:
  Operational data (PDFs, Images, or Voice) originates from the user's device and is transmitted via Base64 encoding to the Node.js backend. The data is processed by the Gemini 2.5 Flash model to extract structured entities, which are then cross-referenced with existing records in Firestore. Finally, conflict alerts and AI recommendations are pushed back to the specific departments involved.
- Model Process:
  The model illustrates an end-to-end intelligent workflow: from automated data extraction (New Record) to natural language machine allocation (Request Machine) and concluding with multi-departmental conflict resolution (Active Conflicts).
- Role of Reference:
  This document serves as a technical blueprint for developers and stakeholders to understand the integration between Generative AI and Role-Based Access Control (RBAC), ensuring consistent implementation of security and privacy rules.

© UMHackathon 2026

1.2. Background

Currently, industrial operations across Production, Maintenance, Logistics, and Quality Control rely on fragmented manual inputs and physical documentation. This leads to scheduling overlaps and resource conflicts that are often only identified when production is already halted.

- **Previous Version**
  a. Manual Spreadsheets: Static files that do not update in real-time.
  b. Fragmented Messaging: Unstructured communication via apps like WhatsApp or email, where critical machine requests are easily lost.
  c. Rule-Based Logic: Simple calendar systems that can detect two things happening at once but cannot understand the context or urgency behind the request.

- **Changes in Major Architectural Components**
  The shift from a manual/rule-based system to OpsResolve introduces three major architectural changes:
  a. **Centralized AI Engine:** Replaced manual verification with a centralized **Gemini AI** service that acts as a 24/7 "Digital Supervisor" to detect overlaps.
  b. **Real-time NoSQL Database:** Transitioned from static logs to **Firebase Firestore**, enabling instant conflict notifications across all departmental devices.
  c. **Unstructured Data Processing:** Integrated a dedicated API layer to handle high-payload file uploads, allowing the system to read images and PDFs directly.

- **New capabilities that are only introduced in this**
  a. **Departmental Data Siloing:** A privacy-first logic where conflicts are only visible to the specific departments involved (e.g., a Production/Logistics conflict is hidden from Maintenance).
  b. **Voice-to-Resource Mapping:** Users can request machinery using natural speech, which the AI converts into technical requirements to find the best available machine.
  c. **Managerial Override & AI Recommendations:** A decision-support system that doesn't just identify problems but offers solved "Accept/Override" paths for Managers to resolve conflicts in one click.

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Department Staff (Data entry)	Submits resource requests and views machine availability schedules.	A simple interface to submit "Urgent Requests" and clear visibility on when they can use a machine.
Supervisor	Bridge between department staff and manager. Schedule a meeting and give an announcement for departments.	Real-time updates on conflict status and clear communication from the AI regarding why a clash occurred.
Manager	The final decision-maker who reviews AI suggestions and either accepts or overrides them.	Accurate "Impact Analysis" from the AI to understand the risks of each decision (e.g., downtime cost).
Development Team	To engineer a secure, scalable architecture integrating Gemini AI with Firebase for real-time operational processing.	Delivery of a high-performance system that accurately parses unstructured data and strictly enforces departmental data privacy.
QA Team	To rigorously validate system logic and identify vulnerabilities in the conflict detection and role-based access workflows.	Assurance that the AI provides reliable recommendations and that sensitive information remains visible only to authorized departmental users.

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview

Type	Details
System	Web-based Enterprise Dashboard.
Architecture	Serverless-Hybrid Architecture utilizing Firebase as the backend-as-a-service (BaaS) and a dedicated Node.js/Python logic layer for the Agentic Workflow.
Core Clients	Entry Page (Role selection), Record Submission Tab, Request Machine Tab, Active Conflicts tab, Conflicts Pop-up small window.

OpsResolve is structured as a centralized system where three user roles—Data Entry, Supervisor, and Manager—interact via a unified React frontend. These clients communicate with a Node.js/Express backend API. The backend orchestrates data flow between Firebase Firestore (for real-time persistence) and Google Gemini AI (serving as the intelligent processing layer).

2.1.2 LLM as Service Layer

Gemini 2.5 Flash is integrated not as a passive module, but as a core Reasoning Service. It handles unstructured data extraction, natural language intent classification for machine requests, and multi-variable conflict analysis.

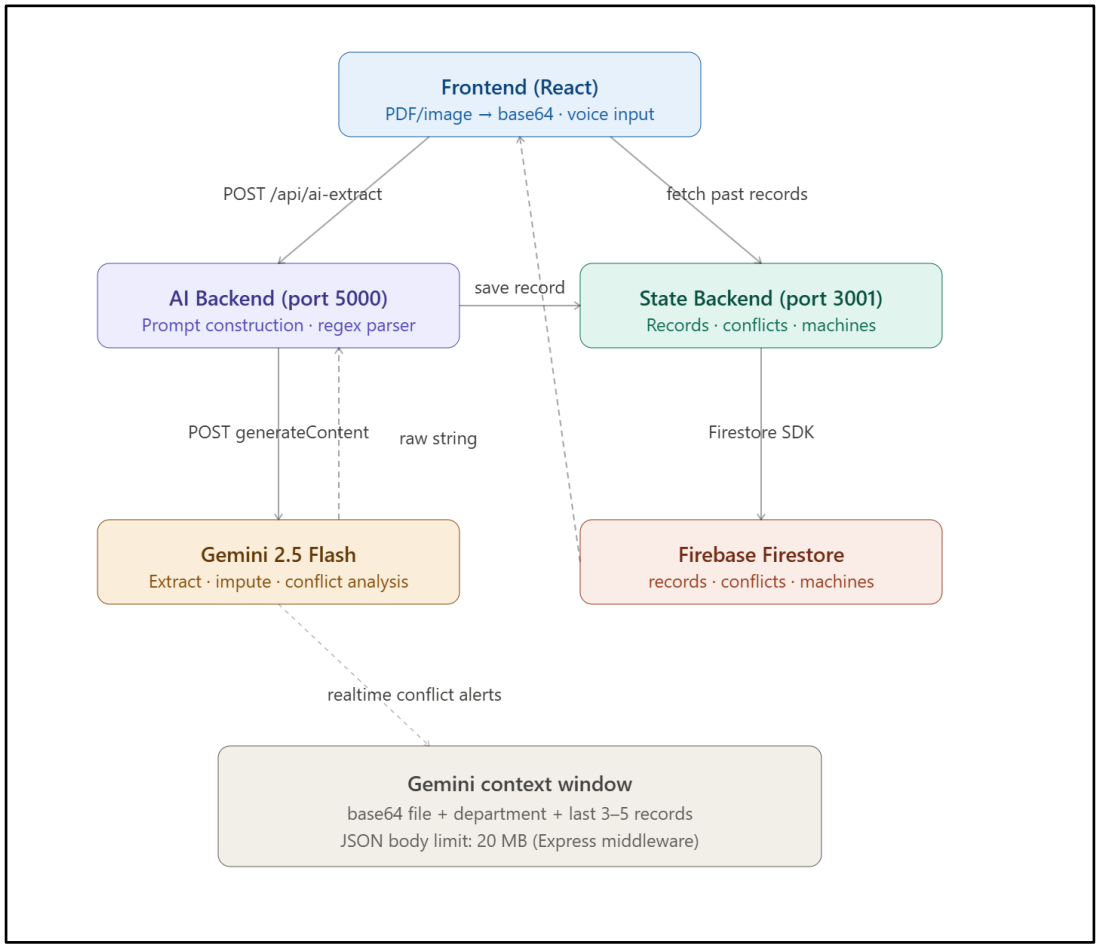
2.1.3 Dependency Diagram

The following outlines the dependencies and data flow involving the LLM:

- **Prompt Construction:** The backend constructs prompts dynamically by combining user inputs (Base64 file strings or voice-to-text) with system instructions and "Past Records" fetched from Firestore to provide few-shot context.
- **Context Window:** The context includes the current uploaded file metadata, the user's department, and a JSON array of the last 3-5 relevant historical records to ensure consistency.

© UMHackathon 2026

- **Parsing & Response:** Gemini returns a raw string (often containing a JSON block). The backend uses a Regex-based parser to strip Markdown (e.g., ```json) and convert the text into a JavaScript object before passing it to the Firestore update service.
- **Token & Input Constraints:** To stay within free-tier limits and optimize performance, images are resized on the frontend, and JSON body limits are enforced at **20MB** via Express middleware.
- **Major API Calls:**
  - **Frontend ↔ Backend:** POST /api/ai-extract for PDF/Image parsing.
  - **Backend ↔ Gemini API:** POST /v1beta/models/gemini-2.5-flash:generateContent.
  - **Frontend/Backend ↔ Firebase:** Real-time listeners via Firestore SDK for conflict alerts.



Dependency diagram

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.1.4 Sequence Diagram

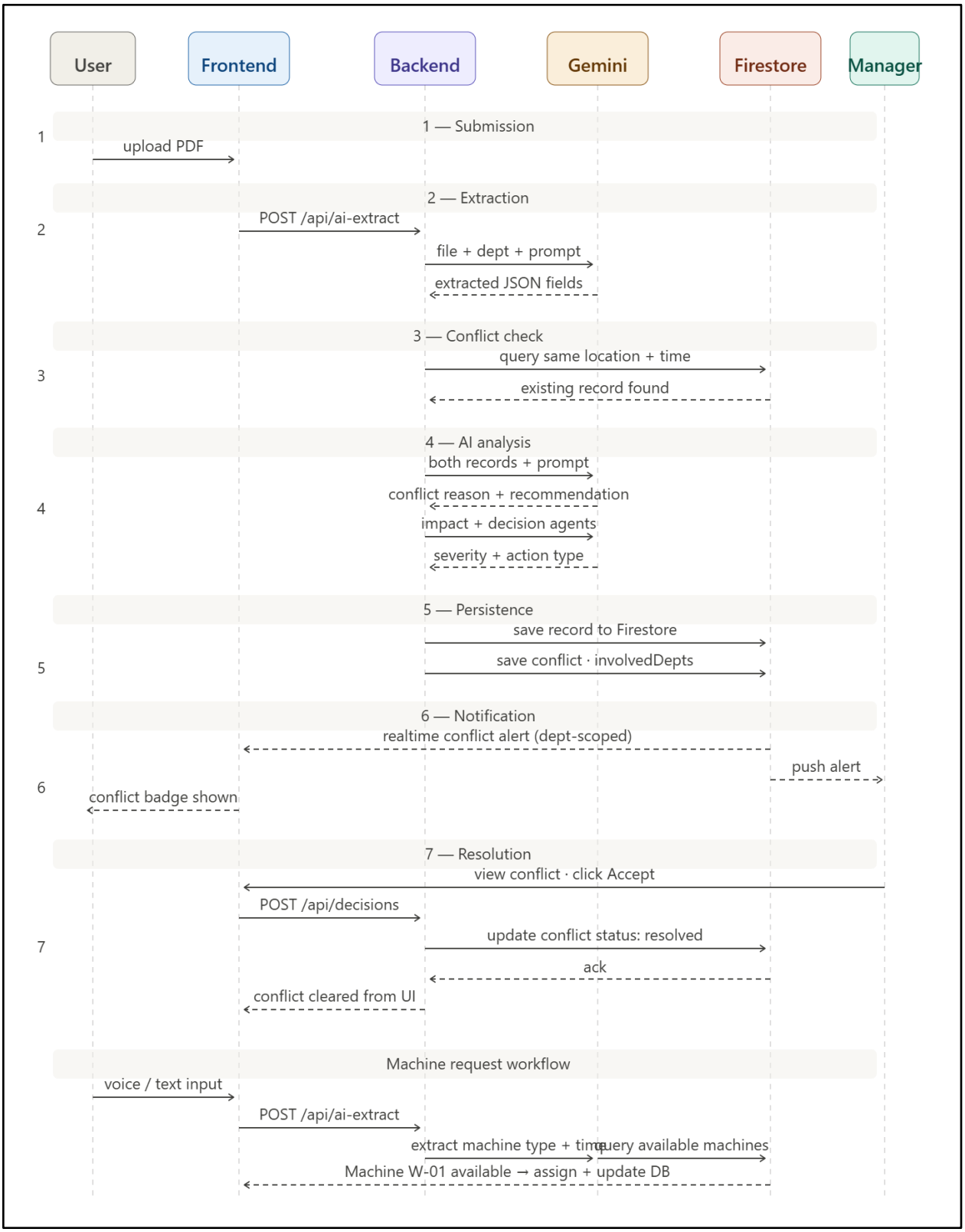
This flow demonstrates the "New Record to Conflict Alert" journey:

1. **Submission:** A **Data Entry** user uploads a PDF schedule for the **Production** department.
2. **Extraction:** The Frontend sends the file to the Backend. The Backend calls **Gemini** to extract the startTime, location, and equipment.
3. **Conflict Check:** The Backend queries **Firestore** for any existing records in the same location during that startTime.
4. **AI Analysis:** If a potential overlap is found, the Backend sends both records to **Gemini** with the prompt: "Analyze these two tasks for a conflict and suggest a resolution."
5. **Persistence:** The Backend saves the conflict into the conflicts collection, tagging it with involvedDepartments: ["Production", "Logistics"].
6. **Notification: Firestore** pushes a real-time update to the **Supervisor** and **Manager** dashboards of the involved departments only.
7. **Resolution:** The **Manager** views the recommendation and clicks "Accept," which updates the record status and clears the conflict.

Model Process: Machine Request Workflow

Step	Component	Action
1. Input	Frontend (Voice)	User speaks: "I need a welding machine for Bay 2 at 3 PM."
2. STT	Web Speech API	Converts voice to a raw text string.
3. Intent	Backend + Gemini	Gemini identifies the machine type (Welding) and time (15:00).
4. Lookup	Firestore	Queries machines collection for status: "Available".
5. Output	Frontend	Displays "Machine W-01 assigned to Bay 2" and updates the DB.

© UMHackathon 2026



Sequence diagram

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2. Technological Stack

2.2.1. Frontend

React.js(Vite): A high-performance web framework chosen for its fast development cycle and responsive UI capabilities. It allows for a dynamic, dashboard-style interface essential for real-time monitoring of machine requests and conflicts across different departments.

2.2.2. Backend

Node.js/Express.js: Serves as the dedicated logic layer. It acts as the orchestrator for the Agentic Workflow, handling complex multi-step processes such as receiving PDF/Voice data, communicating with the LLM, and performing conditional logic before updating the database.

2.2.3. Database

- Firebase Firestore: A NoSQL document database utilized for real-time data synchronization. This ensures that when a conflict is detected by the AI, it is instantly pushed to the Supervisor and Manager dashboards without requiring a page refresh.
- Firebase Auth: Handles secure, role-based authentication (Data Entry, Supervisor, Manager).

2.2.4. Intelligence Layer (LLM)

Google Gemini 2.5 Flash: Integrated as the primary reasoning engine for unstructured data extraction (PDF/Images) and the decision-support system for conflict resolution and recommendation generation.

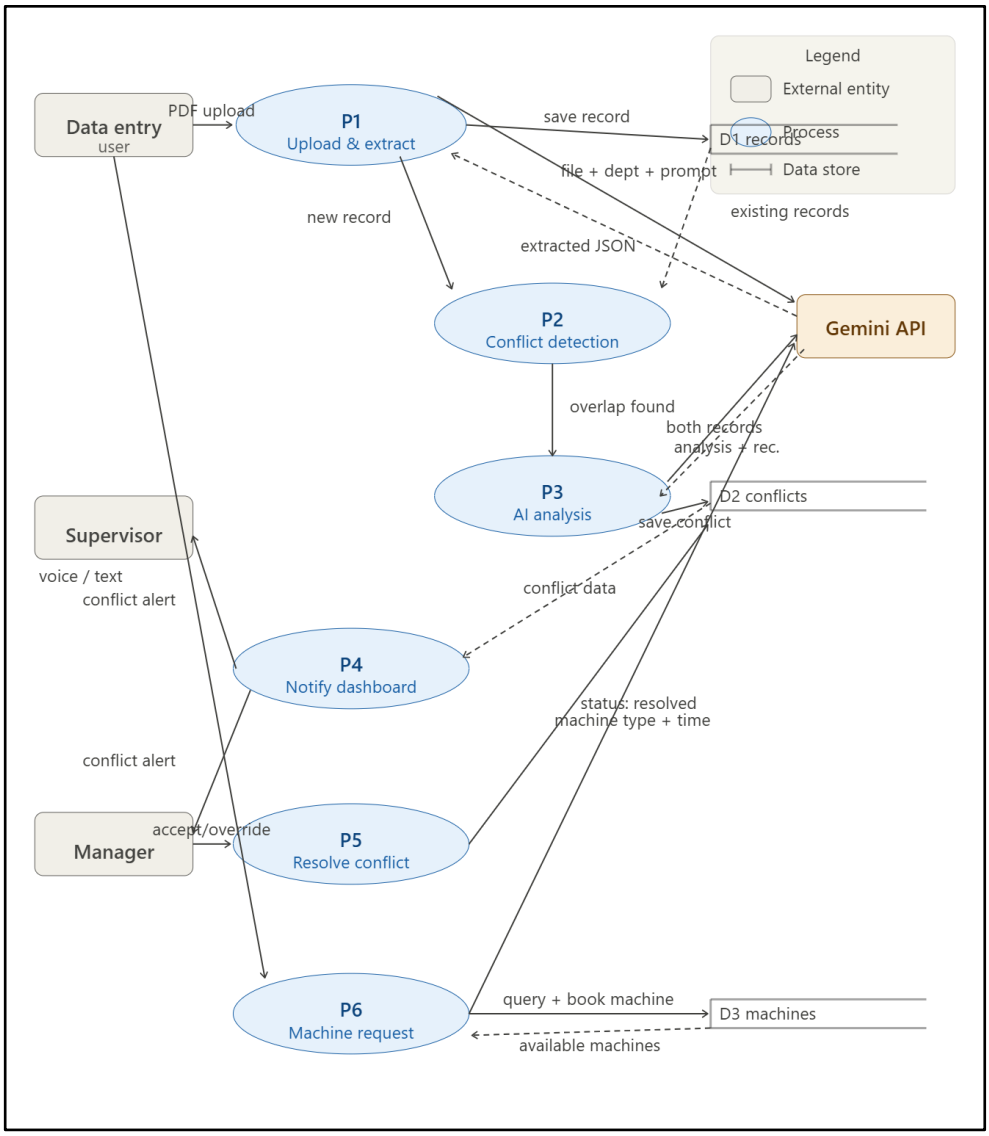
2.2.5. Cloud/Deployment

- Firebase Hosting: For deploying the frontend assets with global CDN support.
- Google Cloud Run / Render: (Or your specific host) Used to host the dedicated Node.js logic layer, providing auto-scaling capabilities to handle peak operational loads during shift changes.
- Firebase Storage: Used for persistent storage of uploaded PDF reports and maintenance images.

© UMHackathon 2026

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)

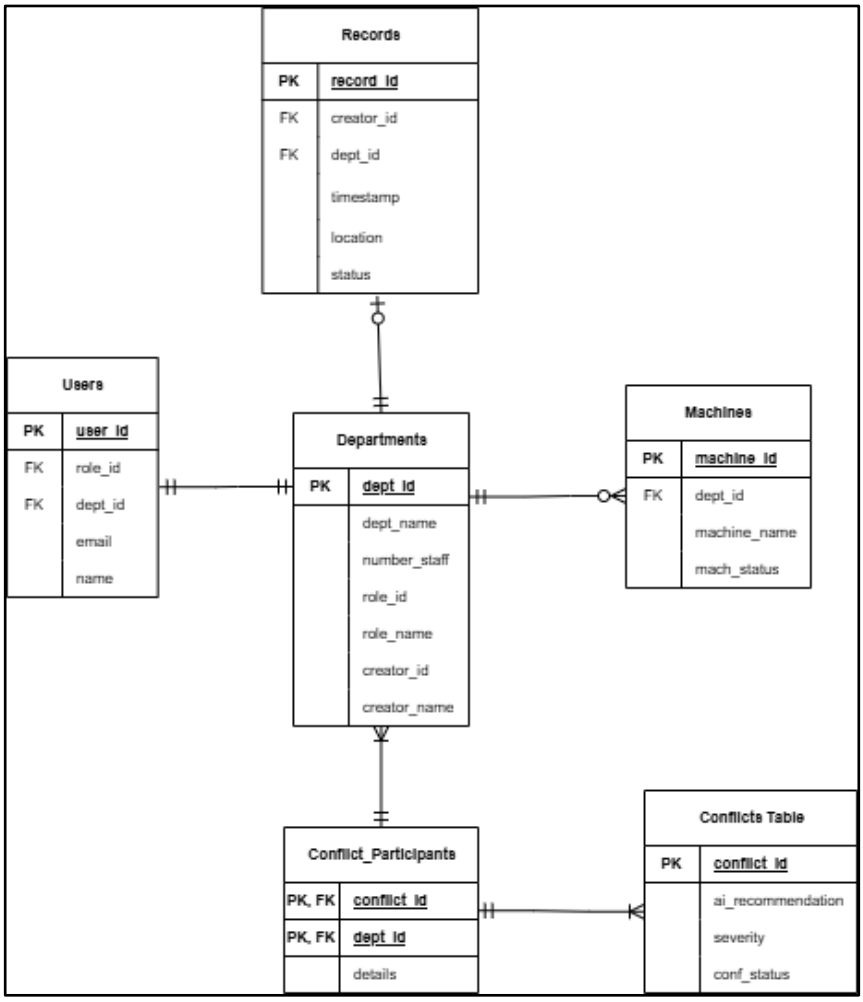


DFD Diagram

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3.2. Normalized Database Schema (ERD Logic)



ERD Diagram (3NF)

3. Functional Requirements & Scope

This section highlights the core boundaries of OpsResolve. By focusing on a high-impact MVP, the team aims to showcase the technical feasibility of integrating Agentic AI workflows with real-time industrial management during UMHackathon 2026.

© UMHackathon 2026

3.1. Minimum Viable Product:

3.1.1. Core Features

No.	Features	Description
1	Multi-Modal Data Extraction	Users upload PDFs, images, or voice notes. The Gemini-powered agent parses these unformatted inputs into structured operational records.
2	Voice-Driven Machine Allocation	Users request equipment via natural language. The system identifies intent and automatically matches requests with available assets in the Firestore database.
3	Agentic Conflict Detection	An autonomous AI agent cross-references new entries against existing schedules to identify resource or time overlaps in real-time.
4	Role-Based Resolution	A specialized dashboard where Managers can "Accept" AI-suggested fixes or "Override" them, while Supervisors trigger departmental notifications.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Privacy	Conflict visibility must be restricted only to the departments involved in the specific overlap.	Use Firestore security rules and "involvedDepartments" array-contains queries for data siloing.
Real-time Sync	Dashboard updates for conflicts must appear within 1 second of detection.	Leverage Firebase's web-socket based onSnapshot listeners for live UI updates.
LLM Reliability	AI extraction must handle varied formatting (e.g., different PDF layouts) with at least 90% accuracy.	Implement "Few-Shot Prompting" by injecting 3-5 historical record examples into the Gemini context window.
Token Latency	AI-generated recommendations must be served to the Manager within 4 seconds of a conflict being identified.	Use the Gemini 2.5 Flash model for its optimized speed-to-

© UMHackathon 2026

		performance ratio and low-latency response times.
Cost Efficiency	Minimize unnecessary API calls to stay within hackathon/free-tier quotas.	Truncate past record history before sending to the LLM and implement local caching for redundant machine-status queries.

3.3. Out of Scope / Future Enhancements

- **Expanded the scope:** Additional departments and roles involved in the system.
- **Predictive Maintenance Alerts:** Using historical data to predict when a machine is likely to fail before a conflict even occurs.
- **External Vendor Integration:** Allowing third-party maintenance contractors to access specific "Maintenance" tickets via a secure external portal.
- **IoT Sensor Integration:** Connecting physical machine sensors directly to the logic layer to update "Machine Availability" status automatically without human input.
- **Multi-Language Voice Support:** Expanding the voice request feature to support local dialects and secondary languages used on the factory floor.

4. Monitor, Evaluation, Assumptions & Dependencies

This section defines the operational safeguards, monitoring strategies, and external factors required to maintain the stability of OpsResolve in a high-stakes industrial environment.

4.1. Technical Evaluation

4.1.1. Grayscale Rollout & A/B Testing

To ensure zero disruption to factory operations, OpsResolve follows a Grayscale Rollout strategy. New features, such as the Voice-Driven Machine Allocation, are initially released to the Maintenance department only (5% of users). Upon successful performance monitoring, the rollout is gradually expanded to Logistics and Production until 100% saturation is reached. A/B Testing is simultaneously conducted on the AI Recommendation UI to determine if "Detailed Reasoning" vs. "Summary Only" leads to faster Manager resolution times.

© UMHackathon 2026

4.1.2. Emergency Rollback (ER) & Golden Release

In the event of critical anomalies (e.g., AI hallucination leading to unsafe schedule overlaps), an Emergency Rollback is triggered. The CI/CD pipeline automatically redeploys the Golden Release—the last verified stable version of the Node.js logic layer and Firestore security rules—to ensure continuous availability of the "View-Only" conflict dashboard.

4.1.3. Priority Matrix

Level	Condition	Action
P1 - Critical	Gemini API downtime or >5% failure in PDF extraction.	Immediate trigger of Emergency Rollback; switch to "Manual Entry" fallback mode.
P2 - High	Conflict notification latency exceeds 10 seconds.	Scale Cloud Run instances and clear Redis cache; notify Admins.
P3 - Medium	Discrepancy in machine availability status between AI and DB.	Trigger a "Sync Agent" to re-verify all machine states; log for review.

4.2. Monitoring

4.2.1. System Health & Monitoring Plan

OpsResolve utilizes an integrated monitoring stack to track system health:

Monitoring Tier	Metrics Tracked	Failure Trigger (Health Damage)
AI Intelligence	Response validity and "hallucination" rate.	Triggered if the AI returns non-JSON or unstructured text more than 3 times in a row.
Real-time Sync	Firebase Firestore listener latency (Time-to-Update).	Triggered if the delay between a record submission and a manager alert exceeds 5 seconds.
Alerting	An automated alert is sent to the Administrator role via Firebase Cloud	Triggered if the system detects a "Black Hole" (data received but not processed within 15

© UMHackathon 2026

	Messaging.	seconds)

4.3. Assumptions

a. All factory floor terminals and managerial devices maintain a stable, persistent internet connection to support Firebase real-time synchronization.
b. Industrial records and machine schedules already stored in the system are accurate and serve as a reliable baseline for the ILMU AI to perform conflict detection.
c. Departments have dedicated hardware (tablets/workstations) to access the web dashboards in high-noise environments.
d. At least one authorized supervisor or manager is active on the dashboard to review conflicts and broadcast resolutions.
e. Uploaded PDFs and images meet a minimum resolution requirement for clear OCR processing by the AI engine.

4.4. External Dependencies

Tools	Purpose	Risks
Google Gemini API	Core reasoning engine for PDF extraction and conflict logic, generating structured resolution recommendations.	High: API rate limits or leaked keys. Mitigation: Implement request queuing and strict .env management.
Firebase BaaS	Real-time database, authentication, and file storage.	Medium: Regional service outages. Mitigation: Enable Firestore multi-region data replication.
Web Speech API	Browser-side conversion of voice requests to text.	Low: Browser compatibility issues. Mitigation: Provide manual text-input fallback in the UI.
Render / Cloud Run	Hosting for the dedicated Node.js logic layer.	Medium: Cold start latency. Mitigation: Maintain a "warm" instance during active factory shifts.

© UMHackathon 2026

5. Project Management & Team Contributions

5.1. Project Timeline

Activity\Day	1	2	3	4	5	6	7	8	9	10
Project Planning										
Core Logic (Sprint A)										
Feature Dev (Sprint B)										
QA & Refinement										
Final Demo Presentation & Submission										

Summary of above Gantt Chart:
Day 1 - 2,Project Planning: SAD Report Task Distribution, ERD Schema, Firebase Environment Config. Establishing a solid architectural foundation prevents "scope creep" and technical debt later.

Day 3 - 5,Core Logic (Sprint A): Gemini AI integration, PDF/Image extraction, Node.js Backend. Prioritizing the "Intelligence Layer" as it is the highest-risk technical component.

Day 6-7,Feature Dev (Sprint B): Conflict Detection Agent, Voice-to-Machine flow, RBAC UI. Building the "Functional" features once the AI extraction logic is stable.

Day 8,QA & Refinement: Security rule testing, Bug fixing, Prompt tuning. Ensuring the "Departmental Silo" privacy logic works perfectly before the demo.

Day 9 - 10,Final Demo Prep: Video demo recording, pitch deck, final deployment. Polishing the presentation to effectively showcase the project's impact and feasibility.

© UMHackathon 2026

5.2. Team Members Role

Member A (Project Manager & AI Architect): Yi Jing
Role: Lead Architect.
Responsibility: Design of the Agentic Workflow and Prompt Engineering. Orchestrates the high-level logic between the LLM reasoning and the system's reaction to conflicts.

Member B (Backend Developer): Yi Xian
Role: Infrastructure Lead.
Responsibility: Manages the Node.js/Express environment and Firebase BaaS (Firestore, Auth, Storage). Ensures the "Serverless-Hybrid" bridge is secure and fast.

Member C (Frontend Developer): Yen Yee
Role: UX/UI Specialist.
Responsibility: Builds the React.js Dashboard. Implements the complex Role-Based Access Control (RBAC) UI, ensuring Managers and Supervisors see only their relevant departmental data.

Member D (AI & Data Engineer): Shu Xian
Role: Intelligence Specialist.
Responsibility: Focused entirely on Data Extraction and STT (Speech-to-Text). Handles the processing of PDFs/Images and ensures the output from Gemini is consistently parsed into clean JSON for the database.

Member E (QA & DevOps Engineer): Angie
Role: Quality & Deployment Lead.
Responsibility: Manages the Deployment Pipeline (Cloud Run/Firebase Hosting) and conducts rigorous Edge-Case Testing. Validates that the "Departmental Silo" logic is unbreakable and that the system recovers gracefully from API timeouts.

5.3. Recommendations

- Scalability (Distributed Logic): As the user base grows, I recommend migrating the Node.js logic layer to Google Cloud Functions (2nd Gen). This allows for massive scaling during peak shift-change hours without maintaining idle server costs.
- Performance (Redis Caching): To improve the latency of the "Request Machine" feature, implement Redis Cache for machine status. Querying a cache is faster than a round-trip to the LLM or Firestore for static asset availability.
- Reliability (Prompt Versioning): Use a tool like LangSmith or a custom versioning system for your prompts. This ensures that a minor change in the instructions doesn't break the PDF extraction logic for older report formats.

© UMHackathon 2026

- Security (App Check): Enable Firebase App Check to ensure that only your verified web application can communicate with your Firestore database, preventing unauthorized API calls that could drain your Gemini token quota.

© UMHackathon 2026